

class: A class is a user-defined data types which allows user to group the activities, data and actions together.

This activities or actions are called function or methods.

- * It is a blueprint or template for creating objects.
- * An object is a real instance created from the class.

Public access specifier:

Whatever data type or functions are declared public can be accessed from that class as well as outside the class.

- * By default, the access specifier in c++ is private for classes.

```
class Test {  
    int x; → private  
}
```

```
struct Test {  
    int x; → public  
}
```

In a class → inheritance is private by default
BOROSIL® structure → public inheritance by
Scientific Date: default.
Borosil Scientific Limited

Why this difference?

- (i) classes are designed for data hiding and encapsulation, OOP principles.
(ii) Structures are meant for simple data grouping.

* When to use a struct:

- (1) You only need to group related data.
- (2) Members should be publicly accessible.
- (3) It represents a single data record.

* When to use a class:

- (1) You need encapsulation.
- (2) Data must be protected or hidden.
- (3) You need to enforce rules using methods.

* Creating Objects in different ways:

```
class Human {
```

```
public:
```

```
    string name;
```

```
    void intro() {
```

```
        cout << "Hello";
```

```
    }  
};
```

```
int main() {
```

```
    Human h1;    ——— (1)
```

creates object dynamically

```
    Human *h2 = new Human();
```

↑

returns the memory address

∴ We need a pointer here

(1) Human h1 (Stack Allocation) :

- * Automatically destroyed when scope ends.
- * Fast & no memory leak risk.
- * Used when an object is small & lifetime is limited to a scope.

(2) Dynamic Object (Heap Allocation using new keyword) :

```
Student *s1 = new Student();
```

or with constructor

```
Student *s1 = new Student(100);
```

- * Object survives outside the scope.
- * Useful for large objects or dynamic lifetime.
- * This object must be deleted manually :
delete s1;

(3) Object with constructor Initialization:

```
student s1(100);
```

This ensures the object is initialized properly.

(4) Object using pointer to stack Object:

```
student s1;
```

```
student *ptr = &s1;
```

No dynamic allocation.

Pointer refers to an existing object.

(5) Array of Objects:

```
Student arr[3];
```

or

```
student arr[3] = { student(1), student(2),  
                  student(3) };
```

Useful when multiple objects are needed.

(6) Copy Constructor:

```
Student s1(100);
```

```
student s2 = s1;
```

7) Using Static keyword:

```
static Student s1;
```

- * Object exists for entire program lifetime
- * Memory allocated once.

8) Object returned from a function:

```
Student createStudent() {  
    Student s1(100);  
    return s1;  
}
```

```
int main() {
```

```
    Student s1 = createStudent();  
}
```

* Scope:

The region of a program where a variable or object is visible and accessible.

It defined where a variable / object can be used in a program.

```
E.x. int main() {  
    cout << y... ;  
}
```

cout << y... → ERROR (y is out of scope)

* Types of scopes:

(1) Local / Block scope:

- * Variables declared inside { }
- * Exists until the closing bracket.

(2) Function Scope:

```
int fun (int x) {  
    x exists only inside fun()  
}
```

(3) Global Scope:

- * Declared outside all functions & class.
- * Accessible throughout the program.

(4) Class scope:

E.x. members of a class.

Accessible outside the class using objects.

Scope Resolution Operator (::)

It is used to define or access something that belongs to a specific scope, especially when multiple scopes contain identifiers with the same name.

(1) Defining member functions outside a class:

When a class function is declared inside the class but defined outside, :: is required.

```
class Human {
```

```
public: String name;
```

```
void introduce();
```

```
};
```

introduce function belongs to Human class

```
void Human :: introduce() {
```

```
cout << Human :: name;
```

```
};
```

Accessing the property of class

(2) Accessing a Global Variable when local variable with same name exists:

When a local variable has same name as a global variable, the local one hides the global one.

* A static member function can be called even if no objects of the class have been declared.

Date: _____

```
int x = 10;

int main () {
    int x = 20;
    cout << x;           → 20 (local)
    cout << ::x;         → 10 (global)
}
    Tells compiler to use global x
    not the local x.
```

(3) Accessing static class members:

Static members belong to the class, not to any object.

We can call a static member function using the class name & scope resolution operator.

```
class Test {
    public:    static int count;
}

int Test :: count = 5;
```

```
int main () {
    cout << Test :: count;
}
```

(4) Using namespaces:

```
namespace A {  
    int x = 5;  
}
```

```
namespace B {  
    int x = 10;  
}
```

```
int main () {
```

```
    std::cout << A::x;    → 5
```

```
    std::cout << B::x;    → 10  
}
```

cout belongs to std namespace.

(5) Accessing class members with same name as global name:

```
int x = 50; int value = 50;
```

```
class Demo {
```

```
    public:    int value = 500;
```

```
    void show () {
```

```
        cout << value;    → 500
```

```
        cout << ::value;    → 50
```

```
    }
```

```
};
```

Private Access Specifier

- * Members declared as private can be accessed only inside the same class.
- * They cannot be accessed:
 - * From main()
 - * From other classes.
 - * from derived (child) classes.
- * Private members are accessible only by the member functions of the same class.

```
class Human {  
    private: int age;  
    public:  
        int getAge() {  
            return age;  
        }  
        void setAge (int x) {  
            age = x;  
        }  
};
```

* Accessing private members:

(1) Access through public members of same class (getters - setters):

```
class Student {  
    private: int marks;  
    public:  
        void setMarks (int m) {  
            marks = m;  
        }  
        int getMarks () {  
            return marks;  
        }  
};
```

(2) Internal access through member functions of same class:

```
class Test {  
    private: int x = 10;  
    public:  
        void show() {  
            cout << x; ==> Internal access  
        }  
};
```

Any function that is a member of the class can access private members.

(3) Access using controllers: Constructors are member functions, so access is allowed.

(4) Access using friend functions:

A friend function is not a class member but it is given permission to access private members.

```
class Demo {
```

```
private: int value = 50;
```

```
friend void showValue (Demo);  
};
```

```
void showValue (Demo d) {
```

```
cout << d.value; → Allowed in C++  
};
```

(5) Access using friend class:

```
class A {
```

```
private: int x = 100;
```

```
friend class B;
```

```
};
```

```
class B {
```

```
public:
```

```
void show (A obj)
```

```
{ cout << obj.x; }
```

```
};
```

* (6) Access using static member functions:

```
class Test {
```

```
private: static int x;
```

```
public:
```

```
static void show () {
```

```
cout << x;
```

```
}
```

```
int Test :: x = 5;
```

* Class Constructor:

A constructor is a special member function of a class that is automatically called when an object is created.

Its main purpose is to initialize the data member of the class.

* Key Features of a Constructor:

(1) Name is same as the class name.

(2) No return type (not even void).

(3) called automatically when an object is created.

(4) Used to initialize variables / acquire resources.

(5) can be overloaded.

Types of constructor :

(1) Default: Constructor with no parameter. The compiler automatically provides a default one * if we do not write a constructor.

(2) Parameterized: Takes arguments.

(3) Copy constructor: Used to initialize an object using another object of the same class.

```
class Demo {  
    int x;  
    public:  
        Demo ( Demo &obj) {  
            x = obj.x;  
        }  
};
```

* Can a class constructor be ~~pub~~ private? : Yes

When a constructor is private:

- (i) Object cannot be created directly using the class name.
- (ii) Objects can be created only in controlled way.

How to Use a private constructor:

* Using a public static member function:

```
class A {
```

```
    private:
```

```
        A () { cout << " . . . . " ; }
```

```
    public:
```

```
        static A createObj () {
```

```
            return A();
```

```
        };
```

```
int main () {
```

```
    A obj = A::createObj();
```

```
}
```

(2) Using a friend function:

```
class A {  
    private: A () { cout << "..."; }  
    friend A create();  
};  
  
A create() {  
    return A();  
}  
  
int main () { A obj = create(); }
```

(3) Using a friend class:

```
class A {  
    private: A () { }  
    friend class B;  
};  
  
class B {  
    public:  
        static A getObj() {  
            return A();  
        }  
};  
  
int main () { A obj = B::getObj(); }
```

* Constructor Overloading: Allows a class to have multiple constructors with different arguments to initialize objects differently.

* Same constructor name (as class name).

* Different no. / type / order of parameters.

* The compiler performs compile-time polymorphism and selects the constructor that :

(i) Matches the no. of arguments.

(ii) Matches the data type.

(iii) Has the best possible match.

Destructors

A destructor is a special member function of a class that is invoked automatically when an object is out of scope or is explicitly deleted.

Features of Destructors:

- (i) Name is same as class name but prefixed with '~' (tilde).
- (ii) No return type (not even void).
- (iii) Takes no parameters.
- (iv) Called automatically.
- (v) Only one ~~constructor~~ destructor per class i.e. destructor cannot be overloaded.

A destructor is called when:

- (1) When an object goes out of scope.
- (2) When delete is used on a dynamically allocated object.
- (3) When the program ends.
- (4) When a temporary object is destroyed.

Static Keyword

The static keyword is used to control the lifetime, scope and sharing of variables and functions.

Its exact meaning depends on where it is used.

(1) Local Variable (inside a function):

- * Created only once.
- * Retains its value between function call.
- * lifetime = entire program.
- * Scope = local to function.

```
void count {  
    static int c = 0;  
    cout << c++;  
}
```

```
int main() {  
    count();    —→ 0  
    count();    —→ 1  
    count();    —→ 2  
}
```

Without static, count would reset to 0 every time.

(2) Data member of the class:

- * Belongs to the class, not individual objects.
- * Shared by all objects.
- * Only one copy exists.

```
class student {
```

```
public: static int total;
```

```
    student() { total++; }
```

```
};
```

cannot initialize this
static variable inside
the class

```
int Student :: total = 0;
```

```
int main() {
```

```
    Student s1, s2, s3;
```

```
    cout << Student :: total; —→ 3
```

```
}
```

It can be initialize only
outside the
class

total is common
to all objects.

static data members cannot be initialized inside a class because they do not belong to individual objects; they belong to the class itself and require a single memory allocation outside the class.

* Static members are not part of an object:
Object doesn't contains static members in its definition.

So the compiler cannot allocate memory for it while defining the class.

(3) Member function:

- * Can access only static members.
- * Can be called using class name.
- * Cannot access non-static variables directly.

```
class Test {  
    static int x;  
    public:  
        static void show() {  
            cout << x;  
        }  
};  
int Test :: x = 10;  
int main() {  
    Test :: show();  
}
```

(4) Static Global Variables & Functions:

Limits ~~visibility~~ visibility to only that source file.

* Private static members can be accessed only:

By: (1) other member functions of the same class.

(2) Friend functions & classes

Just like non-static member functions.

* Static keyword is only for a member functions / variables of a class.

* A friend function is not a member function.

so, a friend function cannot be declared as static.

Friend function

- * A friend function is a special type of function that is not a member of a class but is allowed to access the class's private & protected members.
- * It is used when we want external functions to work closely with a class without making the data public.

Uses:

- (i) To allow external functions to access private data.
- (ii) To enable operator overloading.
- (iii) To avoid making data members public.

```
class Box {  
    private: int length;  
    public: Box (int l) { length = l; }  
    friend void showlen (Box);  
};
```

```
void showlen (Box b) {  
    cout << b.length;
```

```
}  
int main() { Box d(10);  
    showlen(d); —, 10
```

- * Not a member of the class.
- * Declared inside a class, defined outside
- * Can access private & protected members.
- * Since it is not a member, it has no this pointer.

* Can a friend function be private?

→ Yes, but it doesn't matter because:

- (i) It is not a member of the class.
- (ii) So it is not affected by private/public/protected access specifier.

* Encapsulation: Binding of data or methods in a class.

- (i) Provides a secure layer.
- (ii) Hides internal implementation of code and data in a class.
- (iii) Exposes only necessary information to the external world.

Also called as Data hiding.

Protected Access Specifier

* Members declared as protected can be accessed inside the same class and inside derived (child) classes, but not from outside the class.

* It is mainly used in inheritance to allow derived classes to access members of parent class while still hiding them from outside the class.

```
class B {  
    protected: int x;  
    public: B() { x = 10; }  
};
```

```
class Derived : public B {  
    public:  
        void show() {  
            cout << x;  
        }  
};
```

```
int main() {  
    Derived d;  
    d.show();  
}
```

Inheritance

- * Inheritance is a mechanism where one class inherits the data members and member functions of another class.
- * It helps in code reusability, extensibility and maintainability.

```
class Base {  
    public: int x;  
    void show () { cout << . . . . ; }  
};
```

Inheritance Operator

```
class Derived : public Base {  
    public:  
    void display () { cout << . . . . . ; }  
};
```

* Types of Inheritance:

(1) Single Inheritance:

```
class A { };
```

```
class B : public A { };
```

(2) Multilevel: Base → derived → derived.

(3) Multiple: Multiple Base → One derived.

(4) Hierarchical: One Base → Multiple derived.

(4) Hybrid Class Inheritance.

- * Private Members are never directly accessible in derived classes.

	Public	Protected	Private
Same class	✓	✓	✓
Derived class	✓	✓	✗
Outside class	✓	✗	✗

(1) Public Inheritance:

Base member	In Derived class
public	public
protected	protected
private	Not accessible

(2) Protected Inheritance:

Base member	In Derived class
public	protected
protected	protected
private	Not accessible

Private members of a class are not accessible in derived classes.

Date: _____

(3) Private Inheritance:

<u>Base class</u>	<u>Derived class</u>
public	private
protected	private
private	Not accessible

Changing Access Level of Base Class Members in derived class

Even if a base class method becomes protected or private due to inheritance, the derived class can change its access level by re-declaring it using the scope resolution operator.

```
class Base {
```

```
    public: void show() {
```

```
        cout << .....;
```

```
    }
```

```
};
```

show() is
now protected
in class Derived

```
class Derived : protected Base {
```

```
    public:
```

Keyword

```
        using Base :: show();
```

```
};
```

Applies the
access

specifier

where it is placed

Brings show() method from
Base class into Derived

∴ Now show() is public in class Derived.

* Private Members of Base class still remain inaccessible.

* Must use using

Order of Execution of Constructors and Destructors in Inheritance:

- * Constructor Order: Base class constructor is executed first then derived class constructors.
- * Destructor Order: Derived class destructor is executed first, then base class destructor.
- * Why this Order?
 - (i) Base class provides the foundation.
 - (ii) Derived class may depend on base data.
 - (iii) So base must be fully constructed first.
 - (iv) Derived class may use base resources.
 - (v) Derived must clean up before base is destroyed.

Multiple Inheritance

Multiple Base classes → One Derived class

* class Child : public Father, public Mother

* If parent & child class have a method with same name & input parameter, the method will get overridden.

class Father → display()

class Mother → display()

class child → display()

~~class~~ child c;

c.display(); → calls child::display

To call the display() method of class Father or mother, we have to use the scope resolution operator.

c.Father :: display();

c.Mother :: display();

Name hiding problem

If base classes & derived class have methods with same name but different parameters passed, there occurs a problem called Name hiding problem.

```
class Father { display (int h); }  
class Mother { display (double w); }  
class child { display (); }
```

When child defines void display() it hides all display (....) functions of the parent class by default.

so, child c;

c.display(); → ✓

c.display (6); → ✗

c.display (25.4); → ✗

Error:

Not found display(int)
in class child

Error: Not found

display (double)
in class child

Solution: Using the using keyword

```
class child : public Father, public Mother {  
public:  
    using Father::display;  
    using Mother::display;
```

Calling & Passing Values to Base Class Constructors from Derived Class

- (i) Base class constructors are called before the derived class constructors.
- (ii) Base class constructors cannot be called like a normal function.
- (iii) Parameters are passed using an initializer list.

Order of inheritance

```

class child : public Father, public Mother
{
    public:
        child (int h, int w)
            : Father(h), Mother(w) {
            .....
        }
};
    
```

parameter passing

Initializer list

Base class constructors are called in the order of inheritance, not on initializer list.

i.e. even if we write:

```

: Mother(w), Father(x) { ... }
    
```

output will be same because :-

```

class child : public Father, public Mother
    
```

then

* If base class has default constructor:

- (i) compiler automatically calls `Father()`
- (ii) No need to pass anything.

* If base class has only parameterized constructor:

```
class child : public Father {
```

```
public: child() : Father(6) {
```

```
    .....  
    }  
};
```

←
correct way

this keyword

- (i) this is a implicit pointer available inside non-static member functions that points to the object which invoked the function.

Use of this:

- (1) To distinguish data members from parameters with the same name.
- (2) To refer to the current object explicitly.
- (3) To return the calling object.
- (4) To pass the current object to another function.

(1) Referring to the Current Object Explicitly:

```
class Demo {
```

```
    public: void show() {
```

```
        cout << this;
```

```
    };
```

↖ this stores the address of d.

(2) Returning current Object to another function

```
class A {
```

```
    int x = 10;
```

```
    public:
```

```
    void show (A* obj) {
```

```
        cout << obj->x;
```

```
    }
```

```
void call() {
```

```
    show(this);
```

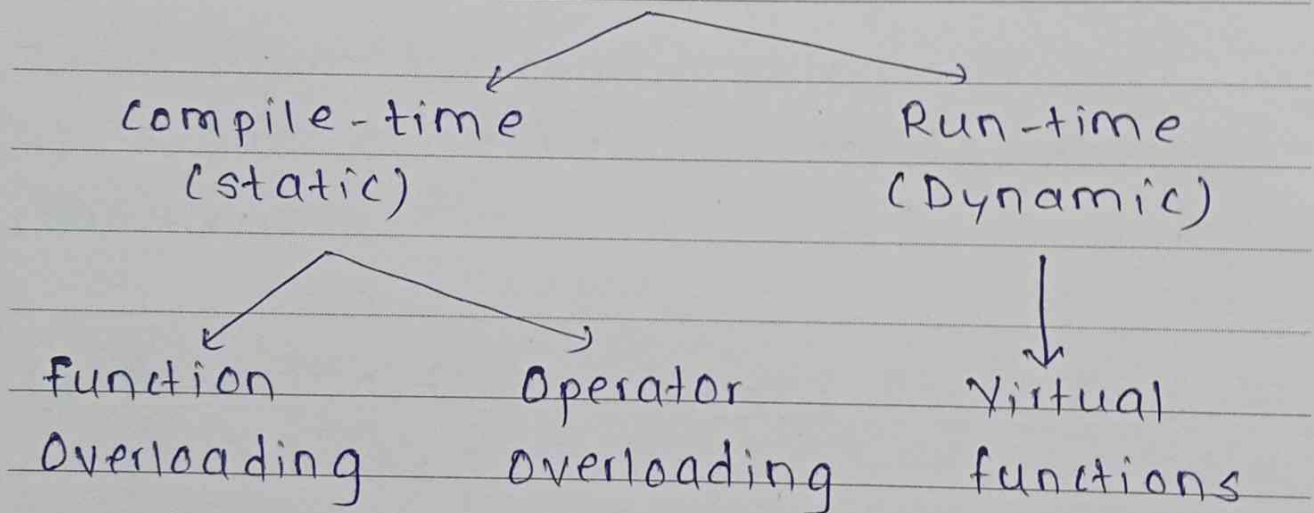
```
    }
```

- * this cannot be used in static functions
- * static members do not belong to any object.
- * No object $\rightarrow$ no this pointer.
- * Every non-static member function receives this implicitly as its first parameter.

Polymorphism

- (i) Polymorphism means - 'many forms'.
- (ii) It refers to the ability of a single function name or operator to behave differently depending on the context -
 - * No. / types of arguments.
 - * Type of object calling it.

Types of polymorphism



(1) Compile-Time (Static Polymorphism):
This is resolved at compile time.

(a) Method Overloading: Same function name but different parameters passed.
The compiler decides which method to call based on arguments.

Function overloading depends only on parameters, not on return type.

Date: _____

(b) Operator Overloading: Giving new meaning to an operator for user-defined types.

(2) RunTime (Dynamic) Polymorphism: Resolved at runtime using inheritance and virtual functions.

* Virtual functions: Without virtual functions, c++ uses compile-time binding (also called static binding).

```
class Person {  
    public: void introduce() {  
        cout << "Base class";  
    }  
};  
        : public Person  
class Student {  
    public: void introduce() {  
        cout << "Derived class";  
    }  
};
```

```
void display (Person &p) {  
    p.introduce();  
}
```

```
int main() {  
    Student s;
```

```
    display(s);
```

Output:

Base class

Even though s points to Student object, the Base version is called.

To solve this, we use 'virtual' keyword.

virtual void show() $\Rightarrow$ Base class

Now, display(s) $\Rightarrow$ Derived class.

$\therefore$ The function call is decided at runtime.

This is runtime polymorphism.

It enables dynamic binding using a mechanism called vtable (virtual table).

How vtable works:

- (i) Each class with virtual functions has a vtable.
- (ii) vtable stores address of virtual functions.
- (iii) Objects stores a hidden pointer (vptr) to its class's vtable.
- (iv) At runtime, the correct function is called using vtable lookup.

* Pure Virtual function: A pure virtual function is a virtual function that has no implementation in the base class.

virtual void display() = 0;

↑
makes the function pure virtual

* It will not have any function body.

* It forces derived classes to implement ~~draw~~ display().

* Abstract Class: A class that contains at least one pure virtual function is called an abstract class.

(i) We cannot create objects of an abstract class.

(ii) But we can create pointers / references of it.

(iii) Derived classes must implement all pure virtual functions of the base class otherwise they will also become abstract.

Override Keyword

Used in derived classes to tell the compiler that a virtual function from the base class is being overridden.

```
class Base {
```

```
public: virtual void show() {
```

```
    cout << "Base class";
```

```
};
```

Function must already be
virtual in base class to
use override

Signature must match

```
class Derived {
```

```
public: void show() override {
```

```
    cout << . . . ;
```

```
};
```

```
class Shape {
```

```
public: virtual void draw() = 0;
```

```
};
```

```
class Circle : public Shape {
```

```
public: void draw() override {
```

```
    cout << . . . ;
```

```
};
```

pointer to the
abstract class

```
int main() { Shape *s1 = new Circle();
```

```
    s1->draw;
```

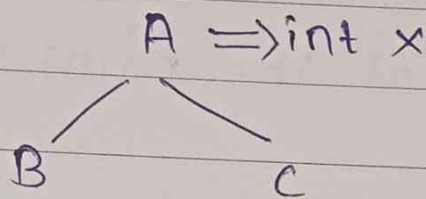
```
}
```

- * Can a pure virtual function be private or protected → Yes
- * Access specifier does not affect abstraction.
- * It only affects how the function call be called, not whether it must be overridden.
- * Access control is checked at the point of CALL, not at the point of Overriding.

Base class Pure Virtual methods	Can Override?	Can call from Derived?
public	✓	✓
protected	✓	✓
private	✓	✗

Virtual Inheritance

- * The diamond problem occurs in inheritance when a class inherits from two classes that have a common base class.
- * This create ambiguity about which base-class members should be used.



```
int main() {  
    D obj;
```

```
    obj.x = 10; —————> Error: ambiguous
```

```
}
```

D has two A subobjects: $B::A$, $C::A$
so $obj.x$ is ambiguous.

Solution: Virtual Inheritance

```
class B : virtual public A { };
```

```
class C : virtual public A { };
```

```
class D : public B, public C { };
```

D obj;
obj.x = 10;

* Ensures only one shared copy of A exists.

* B and C share the same A inside D.

When virtual inheritance is used, the most derived class is responsible for constructing the virtual base.

* Why we cannot create virtual constructors:
Constructors cannot be virtual because virtual dispatch requires a fully constructed object, but constructors run before the object is fully created.

* Virtual functions use:

(1) vptr → inside object.

(2) vtable → function table.

And this happens after object is created.

Nested classes

- * Inner classes exist only to support the outer class.
- * Nested classes do not automatically get access to outer class members. i.e. Inner class cannot access outer class's members directly.

```
class Outer {
```

```
private: int x = 10;
```

```
public: class Inner {
```

```
public: void print(Outer &o) {
```

```
    cout << o.x;
```

```
}; }
```

```
Inner i;
```

```
i.print(Outer);
```

```
};
```

- * creating object of nested class in main()

```
Outer::Inner class obj;
```

```
obj.print();
```

- * Outer class can access all members of the nested class (even private).

* Nested class can have static members and behave like normal class

```
class Outer {  
    class Inner {  
        static void hello() { }  
    }  
};
```

```
int main() { Outer::Inner::hello(); }
```

Local classes

A local class is a class that is defined inside a function or a block. Its scope is limited to that function / block only.

```
void func() {  
    class Local {  
        public: void show() { }  
    };  
    Local obj;  
    obj.show();  
}
```

- (i) Local classes cannot directly access non-static local variables.
- (ii) They can access:
 - * Static local variables
 - * Global variables.
- (iii) Objects can only be created inside that same scope.
- (iv) We cannot create its object outside the function.

Baseclass VTABLE

Base:show

Derived class VTABLE

Derived:show

When you call:

ptr → show();

* ptr points to object d (derived class)

* Object d contains vptr.

* vptr points to Derived's vtable

* compiler looks up show() in that table

* calls Derived::show()

Important clarifications:

①

* `Derived d = new Derived ()`

It is perfectly valid.

* Creating an object of Derived.

* Storing it in a reference of type Derived.

* You can access:

(1) All Derived methods.

(2) All inherited Base methods.

② `Derived d = new Base ()`

This is invalid.

* A parent class object does not have child-specific properties.

* Every Derived is a Base, but not every Base is a Derived.

③ Correct Usage: `Base b = new Derived ();`

* A Derived object is a type of Base.

* So it can be stored in a Base reference.

* But, you can only access methods defined in Base (unless overridden).

* Base b = new Derived();

* Here the object is created dynamically (at runtime) using new.

* Memory is allocated on the heap.

This is also called:

* Runtime Polymorphism

* Upcasting

* Now, How to do this without dynamic allocation:

Derived d; // Stack Allocation

Base *b = &d; // pointing to existing object.

* Runtime Polymorphism:

```
class shape {
```

```
public: void draw() { }
```

```
};
```

```
class Circle : public shape {
```

```
public: void draw() { }
```

```
};
```

```
int main() {
```

```
    Circle c;
```

```
    c.draw() ==> Circle::draw();  
    (As expected)
```

Now we introduce a method:

```
void ShapeDrawing (Shape *s) {
```

```
    s -> draw();
```

```
}
```



why introduce this method

we want to handle all the things
by using the base class reference.

Now, int main() {

```
    Circle c;
```

```
    ShapeDrawing (&c)
```

```
}
```

sending pointer of c



* Runtime Polymorphism:

```
class shape {
```

```
public: void draw() { }
```

```
};
```

```
class Circle : public shape {
```

```
public: void draw() { }
```

```
};
```

```
int main() {
```

```
    Circle c;
```

```
    c.draw() ==> Circle::draw();  
}
```

(As expected)

Now we introduce a method:

```
void ShapeDrawing (Shape* s) {
```

```
    s -> draw();
```

```
}
```

why introduce this method



we want to handle all the things
by using the base class reference.

Now, int main() {

```
    Circle c;
```

```
    ShapeDrawing (&c)
```

```
}
```

sending pointer of c



shapeDrawing (shape *s)

shape *s = sc
(Upcasting)

shapeDrawing (sc) $\xrightarrow{\text{expected}}$ Circle::draw()

(compile-time binding)

actual

Shape::draw()

* Why this happens $\Rightarrow$ Because C++ uses compile time binding by default.

* At compile time :

Base * b

* Compiler only knows : b is a pointer of Base so when it sees : b $\rightarrow$ show();

, it thinks - b is Base class object reference so, I should call Base::show();

* It does not care that b is pointing to a Derived class object.

* With virtual keyword:

```
class Base {
public: virtual void show() { }
};
```

Function

signature must match

This tells compiler to wait & decide at runtime. (Runtime / Late Binding)

So - compiler delays decision.

At runtime, it checks actual obj (Derived) correctly calls Derived::show()

When you use virtual, C++ enables runtime dispatch using : * Virtual table (vtable)
* virtual pointer (vptr)

A vtable is a hidden table created by the compiler for a class with virtual functions. It stores - addresses of virtual functions.

Base vtable:
virtual void show(); $\longrightarrow$ &Base::show()

Every object of a class with virtual functions contains a hidden pointer.

vptr $\longrightarrow$ points to the ~~virtual~~ class's vtable.

Derived table:

Derived::show()

Now instead of calling directly

Base::show(),

compiler generates something like:

b $\longrightarrow$ vptr $\longrightarrow$ show()

* Can a class with pure virtual methods have normal methods: Yes

A class that has at least one pure virtual method becomes an abstract class, but it can still contain:

* Normal non-virtual function

* Virtual functions - with implementation (not pure virtual function)

* Data members.

* Derived class:

Must implement: Pure virtual functions.

Does not need to implement → others.

* If derived class does not implement pure virtual function: It also becomes an abstract class & we will not be able to create its objects.

* We can also call normal methods using base class pointer:

Base *b = new Derived();

b → greet() // Because greet() is already implemented in Base class.

- * Const keyword: Making things read-only
The value cannot be modified after initialization
`const int a = 5;`
 - * Must be initialized at declaration.

* Inline function: An inline function is a function where the compiler tries to replace the function call with the actual function code.

E.x.

```
inline int add(int a, int b) {  
    return a + b;  
}
```

```
int main() {  
    int x = add(2, 3);  
}
```

Instead of a function call, compiler may convert it to;

```
int x = 2 + 3;
```

- * Using inline functions → Avoid function call overhead.